# Tab32 Integration

## 1. Overview

### 1.1 Description

**Tab32 Integration** connects **Tab32** (dental clinic management SaaS) with **Newo Platform**.

It enables:

- Checking available appointment slots across clinic locations
- Booking dental appointments with automatic patient record management
- Cancelling existing appointments
- Multi-location clinic support with intelligent location matching

This integration is designed for **dental clinic administrators and front-desk teams** who need **automated appointment scheduling through a conversational AI agent** (voice or chat).

### 1.2 Use Cases

- When a patient asks about available slots → Check Tab32 availability and return open time slots
- When a patient wants to book an appointment → Search/create patient record and schedule appointment in Tab32
- When a patient wants to cancel an appointment → Cancel the appointment in Tab32
- When a clinic has multiple locations → Agent uses LLM to match patient's preferred location from natural language
- When a new patient books → Automatically create patient record in Tab32 before booking

### 1.3 Supported Features

| Feature | Supported | Notes |
|---|---|---|
| Check Availability | ✅ | Date/time/location-based slot search |
| Book Appointment | ✅ | With automatic patient creation if needed |
| Cancel Appointment | ✅ | Requires booking_id from prior booking |
| Multi-Location Support | ✅ | LLM-powered location matching (Gemini) |
| Single-Location Mode | ✅ | Simplified setup with pre-selected location |
| Patient Search | ✅ | By phone number + name matching |
| Patient Creation | ✅ | Auto-creates during booking flow |
| Timezone Handling | ✅ | Converts slots to clinic's local timezone |
| Reschedule Appointment | ❌ | Cancel + rebook as workaround |
| Historical Data Import | ❌ | Only new events after activation |

## 2. Requirements

Before installation:

- Active **Tab32** account with API access
- **Tab32 API Subscription Key** (PDDS-Subscription-Key)
- Tab32 API base URL (default: `https://openapi.tab32.com/api` )

## 3. How to Setup

### 3.1 Step 1 — Generate API Key in Tab32

1. Log in to your **Tab32** account
2. Navigate to the API management section
3. Generate or locate your **PDDS-Subscription-Key**
4. Copy the key and store it securely

### 3.2 Step 2 — Connect in Platform

1. Open **Newo → Projects**
2. Set the following attributes in **Builder / Attributes**:

| Attribute | Required | Description |
|---|---|---|
| `tab32_api_key` | ✅ | API Subscription Key (PDDS-Subscription-Key) |
| `tab32_base_url` | ❌ | API base URL (default: `https://openapi.tab32.com/api`) |
| `tab32_multilocation` | ❌ | Enable multi-location support (default: `True`) |
| `tab32_slot_duration` | ❌ | Slot duration in minutes: 15, 30, 45, or 60 (default: 30) |
| `tab32_enable_booking` | ❌ | Enable booking capability |
| `tab32_enable_cancellation` | ❌ | Enable cancellation capability |
| `tab32_enable_slot_check` | ❌ | Enable availability check |
| `tab32_check_existing_client` | ❌ | Verify if patient already exists before creating |
| `tab32_enable_client_history` | ❌ | Enable client history tracking |

3. Click **Save + Publish All**
4. On publish, the SetupFlow automatically:
    - Validates the API key
    - Fetches clinic locations from Tab32
    - Configures booking tools for the agent

## 4. How to Use

This section explains how the integration works, how to configure automation, and how to test it.

### 4.1 How the Integration Works

The integration operates based on **Triggers and Actions** via the event-driven system.

- A **Trigger** is a system event that starts a flow
- An **Action** is the API operation performed in Tab32

**Available Triggers**

| Trigger | Event | Description |
|---|---|---|
| **Check Availability** | `get_available_slots` | When the agent needs to look up open slots |
| **Book Appointment** | `book_slot` | When the agent confirms a booking |
| **Cancel Appointment** | `convo_agent_cancel_booking` | When the agent processes a cancellation |
| **Session Start** | `session_started` | Preloads offices context for the session |
| **Setup** | `project_publish_finish` | Initializes integration on deploy |

**Available Actions**

- **Search Available Slots** — Queries Tab32 for open appointment times at a location
- **Search Patient** — Finds existing patient by phone number
- **Create Patient** — Creates a new patient record (first name, last name, phone, email, DOB, gender)
- **Create Appointment** — Books an appointment slot (date, time, location, patient)
- **Cancel Appointment** — Cancels an existing appointment by booking ID

## 4.2 Architecture Diagrams

**Overall Sequence**

```
sequenceDiagram
    participant U as User
    participant A as ConvoAgent
    participant I as Tab32 Integration
    participant API as Tab32 API

    Note over I: Setup (on publish)
    A->>I: project_publish_finish
    I->>API: GET /locations
    API-->>I: Offices list
    I->>A: Booking tools registered

    Note over U,API: Availability Check
    U->>A: "Any slots on Monday?"
    A->>I: get_available_slots (date, location)
    I->>I: LLM location matching (multilocation)
    I->>API: GET /appointments/search-available-appointment
    API-->>I: Available slots
    I->>I: Convert to local timezone
    I->>A: result_success (availability[])
    A->>U: "Here are the available times..."

    Note over U,API: Booking
    U->>A: "Book 10:00 AM please"
    A->>I: book_slot (customerInfo, dateTime, location)
    I->>API: GET /search-patients (phone)
    API-->>I: Patient results
    alt Patient not found
        I->>API: POST /patient (name, phone, email, DOB)
```

```
        API-->>I: New patient_id
    end
    I->>API: POST /appointments (slot, patient_id, location)
    API-->>I: appointment_id
    I->>A: result_success (booking_id, patient_id)
    A->>U: "Your appointment is confirmed!"

    Note over U,API: Cancellation
    U->>A: "Cancel my appointment"
    A->>I: convo_agent_cancel_booking (booking_id)
    I->>API: POST /appointments (status: CANCELLED)
    API-->>I: Confirmation
    I->>A: result_success (cancelled)
    A->>U: "Your appointment has been cancelled"
```

**AvailabilityFlow**

```
flowchart TD
    E["get_available_slots"] --> EX["Extract payload:<br>date, time, location"]
    EX --> ML{"Multilocation?"}
    ML -->|"Yes"| LLM["LLM match location<br>(Gemini)"]
    LLM --> FOUND{"Location<br>matched?"}
    FOUND -->|"No"| FB["Fallback:<br>ask user"]
    FOUND -->|"Yes"| VAL["Store location_id<br>in persona"]
    ML -->|"No"| PRE["Use pre-selected<br>location"]
    PRE --> VAL
    VAL --> CHK{"Date<br>provided?"}
    CHK -->|"No"| FB
    CHK -->|"Yes"| API["GET /search-available-appointment<br>{location_id, date,
duration}"]
    API --> OK{"HTTP 200?"}
    OK -->|"No"| ERR["ResultError"]
    OK -->|"Yes"| TZ["Convert slots<br>to local timezone"]
    TZ --> CACHE["Cache slots<br>in persona"]
    CACHE --> OUT["result_success<br>{availability[]}"]
```

**BookingFlow**

```
flowchart TD
    E["book_slot"] --> EX["Extract customerInfo:<br>name, phone, email, DOB"]
    EX --> LOC["Resolve location_id"]
    LOC --> VAL{"phone +<br>name valid?"}
    VAL -->|"No"| FB["Fallback:<br>ask user"]
    VAL -->|"Yes"| SEARCH["GET /search-patients<br>{phone}"]
    SEARCH --> OK1{"HTTP 200?"}
    OK1 -->|"No"| ERR["ResultError"]
    OK1 -->|"Yes"| MATCH{"Patient found?<br>(name match)"}
    MATCH -->|"Yes"| USE["Use existing<br>patient_id"]
    MATCH -->|"No"| CREATE["POST /patient<br>{name, phone, DOB}"]
    CREATE --> OK2{"HTTP 201?"}
```

```
    OK2 -->|"No"| ERR
    OK2 -->|"Yes"| NEW["Extract new<br>patient_id"]
    USE --> BOOK
    NEW --> BOOK
    BOOK["POST /appointments<br>{patient, slot, location}"] --> OK3{"HTTP 201?"}
    OK3 -->|"No"| ERR
    OK3 -->|"Yes"| OUT["result_success<br>{booking_id, patient_id}"]
```

**CancellationFlow**

```
flowchart TD
    E["convo_agent_cancel_booking"] --> VAL{"booking_id<br>present?"}
    VAL -->|"No"| FB["Fallback:<br>cancellation failed"]
    VAL -->|"Yes"| API["POST /appointments<br>{status: CANCELLED}"]
    API --> OK{"HTTP 200?"}
    OK -->|"No"| ERR["ResultError"]
    OK -->|"Yes"| OUT["result_success<br>{cancel_booking}"]
```

### 4.3 Where to Test

Testing can be done through the **Newo conversation interface** (chat or voice channel) by interacting with the agent. API responses are logged via the integration's HTTP connector.

### 4.4 Testing and Verification

**To test the integration:**

1. **Setup**: Publish the project and verify logs show successful location fetch
2. **Availability**: Ask the agent "What slots are available on Monday?" — verify slots returned from Tab32
3. **Booking**: Complete a booking conversation — verify appointment appears in Tab32 dashboard
4. **Cancellation**: Request cancellation of a booked appointment — verify status changed in Tab32

**If no action occurs:**

- Ensure `tab32_api_key` is set correctly
- Verify `tab32_base_url` points to the correct endpoint
- Confirm the integration was re-published after configuration changes
- Check that the relevant feature flags are enabled ( `tab32_enable_booking` , `tab32_enable_slot_check` , `tab32_enable_cancellation` )
- In multi-location mode: verify `tab32_offices` was populated during setup

> *Note: This integration performs real operations in Tab32. Appointments created or cancelled through the agent are reflected in the live Tab32 system.*

## 5. FAQ

**Q: Does the integration import historical appointments?** A: No. Only appointments created or managed after activation are processed.

**Q: Can I connect multiple Tab32 accounts?** A: Each integration instance supports one Tab32 API key. For multiple accounts, create separate integration instances.

**Q: What happens if a patient already exists in Tab32?** A: The booking flow searches for existing patients by phone number and name. If a match is found, the existing patient record is used. If not, a new patient is created automatically.

**Q: How does multi-location mode work?** A: When enabled ( `tab32_multilocation=True` ), the agent uses an LLM (Gemini) to match the patient's natural language location description to the available clinic offices. The matched location is cached for the session.

**Q: What slot durations are supported?** A: 15, 30, 45, or 60 minute slots, configured via the `tab32_slot_duration` attribute. Default is 30 minutes.

**Q: Why are availability times shown in a different timezone?** A: All times are converted to the clinic location's local timezone. Verify that the correct timezone was imported during setup by checking the `location_time_zone` attribute.